Multi-Period LLM Query Routing

Milestone 4: Complete Optimization Model | Team 32 (T)

Jingyuan Chen, Yi-Nung Hwang, Alan Porhel, Yaqi Wang, Tianyi Zhang
IEOR E4004 – Prof. Yaren Bilge Kaya | November 2026

1. Problem Overview

An AI-service operator must allocate query demand among three language models over four six-hour periods. The plan minimizes expected error-score loss, historical API spending and changes in the routing rule, subject to demand satisfaction and a per-period spending budget. This is aggregate routing, not model training or an oracle that knows each answer in advance.

We retain M3's model pool, four task categories, five equally weighted bootstrap scenarios, continuous routing variables, operational-loss objective and adjacent-period coupling. At the original baseline, loosening the budget or removing the switching penalty left the allocation unchanged (Section 4.4). M4 therefore makes two explicit modeling refinements: (i) the budget caps *total* spend per period instead of each model separately, and (ii) the switching penalty measures changes in routing *shares* instead of routed volume. The M3 version remains in the code as an option and is reported for comparison. No newer LLMs are substituted.

2. Data and Parameters

Source: Martian RouterBench, `routerbench_0shot.pkl` [1]. The pinned source contains 36,497 rows. The selected benchmarks contain 2,361 rows; removing two exact duplicate prompts leaves 2,359 unique prompts. All three model scores and costs are finite; scores lie in $[0,1]$ and costs are nonnegative.

M3 category	Benchmark mapping chosen for M4	Calibration / heldout
Math	MMLU elementary mathematics	264 / 114
Coding	MBPP	297 / 128
Commonsense	WinoGrande	886 / 381
Reasoning	MMLU formal logic + logical fallacies	202 / 87

M3 names Reasoning but does not specify its benchmark. We use formal logic (126 rows) and logical fallacies (163 rows) as an explicit operational definition chosen by task content, not by observed model rankings.

A category-stratified 70/30 split (seed 4004) yields 1,649 calibration and 710 heldout rows. Each of five scenarios resamples calibration rows with replacement within each category, using the same sampled rows for every model and for both score and cost:

$$q_{kms} = \frac{1}{|I_{ks}|} \sum_{i \in I_{ks}} a_{im}, \qquad c_{kms} = \frac{1}{|I_{ks}|} \sum_{i \in I_{ks}} b_{im},$$

where I_{ks} is the resampled multiset, a_{im} the recorded score and b_{im} the recorded USD cost. Heldout scores never enter estimation. Costs describe historical benchmark executions, not current API prices. Heldout category counts define simulated demand, not measured production traffic.

3. Mathematical Formulation

Symbol	Definition / units
K, M, T, S	K={math, coding, commonsense, reasoning}; $\lvert M\rvert = 3$; $T = \{1,2,3,4\}$; $S = \{1,\ldots,5\}$.
Model pool M	gpt-3.5-turbo-1106; gpt-4-1106-preview (r, the reference model); mistralai/mixtral-8x7b-chat.
q_{kms}, c_{kms}	Scenario mean score in $[0,1]$; historical USD per query.
d_{kt}, p_s	Category demand in period t (queries); scenario weight $p_s = 1/5$.
B_{ts}, α	Total budget for period t in scenario s (USD); $\alpha \in [0,1]$ = share of all-GPT-4 spend allowed.
λ, β	Loss units per USD; loss units per demand-weighted L1 routing-change unit.
$x_{kmt} \geq 0$	Continuous query volume of category k routed to model m in period t.
$w^+_{kmt}, w^-_{kmt} \geq 0$	Increase / decrease in the routing share x_{kmt}/d_{kt} versus $t-1$; defined for $t = 2,3,4$.

3.1 Objective Function

$$\min_{x,w^+,w^-} \sum_{s\in S} p_s \sum_{k\in K}\sum_{m\in M}\sum_{t\in T}\left[(1-q_{kms})+\lambda c_{kms}\right]x_{kmt} \;+\; \beta \sum_{k\in K}\sum_{m\in M}\sum_{t=2}^{4} d_{kt}\,(w^+_{kmt}+w^-_{kmt})$$

The first term is expected error-score loss; λ converts API dollars into the same loss units ($\lambda = 10$: \$0.10 of spend is as bad as one wrong answer). The second term is demand-weighted L1 distance between routing shares. A transfer counts both departure and arrival; half this measure is a minimum redistribution proxy, not observed unique requests. Because the plan is common to all scenarios and $\sum_s p_s = 1$, the switching term is counted once.

3.2 Demand Constraint

$$\textstyle\sum_{m\in M} x_{kmt} = d_{kt} \qquad \forall k \in K,\ t \in T$$

Every query is served; none can be dropped. With N_k the heldout count, non-coding demand is $N_k/4$ per period; coding receives 45% of its volume in period 2 and 55%/3 in each other period (M3's demand spike).

3.3 Budget Constraint

$$\textstyle\sum_{k\in K}\sum_{m\in M} c_{kms}\,x_{kmt} \le B_{ts} \quad \forall t \in T,\ s \in S, \qquad B_{ts} = \alpha \sum_{k\in K} c_{krs}\,d_{kt}$$

Total spend in each period may not exceed α times what that period would cost if every query went to GPT-4. Requiring the cap in all five scenarios makes the plan robust to the sampled cost uncertainty. Lowering α lowers the permitted caps and shrinks the feasible set; actual optimal spending need not decrease monotonically in general.

4. Assumptions and Revisions

4.1 Switching Constraint

$$\frac{x_{kmt}}{d_{kt}} - \frac{x_{km,t-1}}{d_{k,t-1}} = w^+_{kmt} - w^-_{kmt} \quad \forall k,m,\ t = 2,3,4; \qquad x_{kmt} \ge 0,\ \ w^+_{kmt}, w^-_{kmt} \ge 0.$$

Because $d_{kt} > 0$ is data, the constraint stays linear. For $\beta > 0$ an optimum has $w^+ + w^- = |\Delta\text{share}|$; otherwise both auxiliaries could be reduced. A demand spike that is served with an unchanged rule is *not* penalized, unlike M3's volume-based choice, which can also charge workload growth. No period-0 plan is given, so period 1 carries no penalty. The LP has 48 routing variables, 72 change auxiliaries and 72 constraints (16 demand, 20 budget, 36 coupling); all coefficients are fixed before solving.

Assumption	Interpretation and limitation
One common plan	x has no scenario index: the plan is fixed before uncertainty resolves, with no recourse.
Sampled uncertainty	Expected-loss objective with budgets enforced in all five sampled scenarios; no guarantee for unseen scenarios.
Continuous volumes	Fractions are expected throughput or routing shares; independent rounding need not preserve feasibility.
Budget reference	Budget scales with the period's workload priced at GPT-4 rates, so the coding spike also raises the period-2 cap.
Fixed model performance	Historical category scores are assumed relevant; latency and rate limits are excluded (no data).

4.2 Parameter Settings

Base case: $\alpha = 0.30$, $\lambda = 10$, $\beta = 0.10$, with the split seed and five scenarios above. These are transparent operating choices, not fitted preferences. α is set where the budget actually binds: $\lambda = 10$ alone already brings spend down to 29% of the all-GPT-4 level (Section 4.3). The tested instance is infeasible at $\alpha = 0.03$ and feasible at 0.034 and 0.035; these checks bracket the feasibility boundary rather than prove a cutoff at 0.035.

4.3 Choosing λ

Without a budget the objective is linear and each category only has to meet its demand, so each category goes entirely to the model with the smallest per-query loss $(1-\bar{q}_{km})+\lambda\bar{c}_{km}$, where $\bar{q}_{km}=\sum_s p_s q_{kms}$ and $\bar{c}_{km}=\sum_s p_s c_{kms}$. Comparing GPT-4 ($G$) with a cheaper model C:

$$(1-\bar{q}_G)+\lambda\bar{c}_G<(1-\bar{q}_C)+\lambda\bar{c}_C \iff \lambda<\frac{\bar{q}_G-\bar{q}_C}{\bar{c}_G-\bar{c}_C}=\lambda^*.$$

Thus λ^* is the accuracy GPT-4 gains per extra dollar and $1/\lambda^*$ its price per avoided error; the upgrade is worth it only when $\lambda<\lambda^*$.

Category (vs. model)	λ_k^*	$1/\lambda_k^*$ ($)
Coding (GPT-3.5)	1.8	0.56
Reasoning (Mixtral)	126	0.008
Math (GPT-3.5)	301	0.003
Commonsense (GPT-3.5)	497	0.002

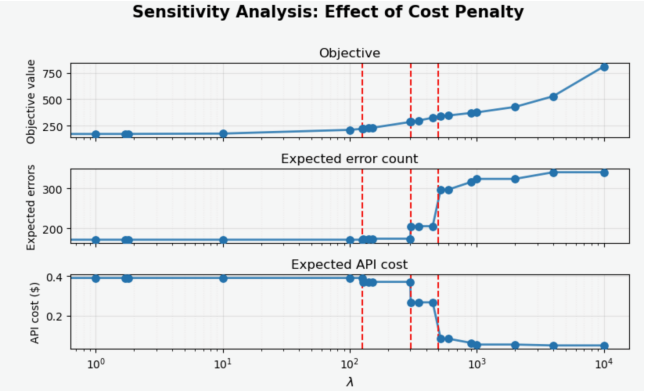


Figure 1: Break-even λ_k^* (left; GPT-4 vs. the model shown, $1/\lambda_k^*$ in USD per avoided error) and effect of λ on the objective, expected errors and cost (right; other parameters at base values; dashed lines mark λ_k^*).

GPT-3.5 is never chosen for reasoning, where Mixtral is both cheaper and more accurate. The unconstrained plan is therefore piecewise constant in λ, and $\lambda=10$ lies inside the plateau $(1.8, 126)$, so any value in that range gives the same routing. At $\alpha=0.30$ the budget binds for every λ up to about 126, so the budget, not λ, fixes the plan: $\lambda=0$ and $\lambda=10$ differ only in the objective value. Figure 1 shows errors and cost flat up to $\lambda\approx 126$ and stepping at the break-even values above it.

4.4 Changes from M3

Element	M3 version and what the real data showed	M4 refinement
Budget	One cap per model, $B_{mts}=\alpha\sum_k c_{kms}d_{kt}$. At $\alpha=0.70$ the plan equals $\alpha=1.0$ (objective 167.145); tightening to $\alpha=0.40$ *raised* spend from \$0.470 to \$0.669.	One cap on total spend per period (3.3); tighter α reduces the allowed total cap; spend falls in the reported sweep.
Switching	Penalized $\lvert x_{kmt}-x_{km,t-1}\rvert$. Demand changes force 68.27 units of change, so $\beta=0$ and $\beta=0.02$ give the same plan.	Penalize share changes weighted by d_{kt} (4.1); β now changes the plan (Section 5).

5. Results

Gurobi returned OPTIMAL on the 2,359-row dataset; SciPy/HiGHS solved the same LP to the same objective (difference below 10^{-9}).

Base-case quantity ($\alpha=0.30,\ \lambda=10,\ \beta=0.10$)	Result
Simulated demand / calibration expected score	710 queries / 75.91%
Expected error-score loss / expected API cost / switching measure	171.04 / \$0.391 / 0
Objective: errors $+10\times$cost $+0.10\times$switching	174.957
Heldout expected score / historical cost	74.68% / \$0.388
Binding budgets (worst-cost scenario) and shadow prices	periods 1, 3, 4: 206, 146, 117 loss units per USD
Maximum demand residual / budget violation	0 / \$0

5.1 Optimal Allocation

Category	Routing rule (same in all four periods)	Volumes in periods 1, 2, 3, 4
Math	100% GPT-4	28.5 each period
Coding	100% GPT-3.5 (GPT-4 is 28× more expensive for +1.6 points)	23.47, 57.60, 23.47, 23.47
Commonsense	100% GPT-4	95.25 each period
Reasoning	20.2% GPT-4, 79.8% Mixtral	4.40 / 17.35 each period

The budget is spent where GPT-4's accuracy gain per dollar is highest (math and commonsense); reasoning absorbs the remaining shortfall. Against the heldout data, this plan scores 74.68% at \$0.388, versus 76.34% at \$1.602 for all-GPT-4 (which violates the budget), 57.18% for all-GPT-3.5 and 53.52% for all-Mixtral. These are category-level expected allocations, not deployment accuracy.

5.2 Sanity Checks

Direct recalculation checks nonnegativity, every demand balance, every sampled budget and the objective (reconstruction gap $\approx 3 \times 10^{-14}$). Hand-solved cases test the quality optimum, a binding total budget with its shadow price, the cost trade-off, infeasibility, that demand growth with an unchanged rule is not penalized, and a β threshold (objective 5 / 7 / 10 for β = 0 / 0.2 / 1). The optimum is never worse than any budget-feasible single-model baseline.

Case (others at base)	Objective	Exp. errors	Exp. cost	Heldout score
Base	174.96	171.04	\$0.391	74.68%
$\beta = 0$: reasoning jumps to 100% GPT-4 in period 2 only (routing-change L1 = 69.4)	172.66	168.55	\$0.411	74.85%
$\lambda = 0$: same plan in this tested comparison	171.04	171.04	\$0.391	74.68%
$\alpha = 0.25$ (tighter)	189.83	186.55	\$0.328	72.83%
$\alpha = 0.40$ or 1.0 (budget slack; base λ, β retained)	165.78	161.08	\$0.470	75.35%
M3 formulation, $\alpha = 0.70$, $\beta = 0.02$ (earlier draft)	167.15	161.08	\$0.470	75.35%

Without switching penalties, routing-change L1 is 69.41 (minimum redistribution proxy 34.71 across boundaries), saving 2.49 expected errors. Budget shadow prices are *additional* scarcity prices; they do not replace λ. With budget multipliers $\mu_{ts} \geq 0$, the Lagrangian's API-cost coefficient for x_{kmt} is

$$\lambda \sum_s p_s c_{kms} + \sum_s \mu_{ts} c_{kms}.$$

A shadow price is the local reduction in the full optimum per extra dollar of one budget RHS. The same plan at $\lambda = 0$ and 10 is an observed sensitivity result, not a general consequence of binding budgets. M5 will sweep α and λ jointly.

5.3 Oracle Comparison

The heldout quality-only oracle picks the best observed score among the three models for each question: 87.04%, versus 74.68% routed (gap 12.36 points). It uses hindsight and ignores budget, cost and switching, so it is a quality upper bound, not operational-objective regret or a feasible policy.

6. Code and Data

6.1 Repository

Code and data:
`https://github.com/GillianChen0323/IEOR4004-LLM-Routing`
The package makes no live LLM calls and needs no API key.

File	Purpose
router_model.py / config.json	Parameter estimation, LP (M4 default; M3 version via `budget_mode="per_model"`, `switch_mode="volume"`), solve, checks, settings.
data/routerbench_selected.csv	2,359 unique rows: IDs, benchmark, category, three scores and three costs.
data/provenance.json / rebuild_data.py	Download URL, source and CSV hashes, mapping and optional reconstruction.
results/	Allocations, budgets with shadow prices, baselines, sensitivity preview, scenario estimates, split IDs, LP and solution files, summary and validation.
test_model.py / test_results.txt	Hand-solvable tests with derivations in comments, and recorded results.

6.2 Re-run from a clean folder

Install `requirements.txt` (Python 3.12), then run `python router_model.py --out results` and `python test_model.py`. Paths are relative to the script. Gurobi's restricted pip license suffices for this 120-variable LP. Tested package versions are in `environment.txt`; file hashes are in `SHA256SUMS.json`. The optional rebuild downloads the $\sim$95 MiB source and verifies its hash before reading it.

6.3 Limitations

The heldout split is a development check, not an untouched benchmark after sensitivity inspection; M5 should use nested resampling. The budget reference ties the period-2 cap to the coding spike; a fixed dollar budget is a natural alternative to test. No integer-rounding guarantee is claimed.

7. AI Policy Disclosure

Claude was used during Milestone 4 to help review the formulation, debug and test code, and format the report in LaTeX. All modeling decisions, parameter choices, results and explanatory writing were verified and finalized by the student authors.

References. [1] Q. J. Hu et al. (2024). RouterBench: A Benchmark for Multi-LLM Routing System. arXiv:2403.12031. Data: `https://huggingface.co/datasets/withmartian/routerbench` (0-shot; DOI 10.57967/hf/1996; accessed November 2, 2026). [2] Team 32, Milestone 3, October 21, 2026; TA feedback, October 15–16; Tianyi's team progress note, October 29. [3] IEOR E4004 Course Project Guide, Fall 2026. Underlying benchmarks: Hendrycks et al., MMLU (2021); Austin et al., MBPP (2021); Sakaguchi et al., WinoGrande (2019); full citations and BibTeX accompany the code.